6. Strings, Runes, Bytes, and UTF-8

06. Strings, Runes, Bytes, and UTF-8

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `06-strings-runes-bytes`

Learning objectives

- Explain that Go strings are immutable byte sequences (often UTF-8)
- Iterate by rune vs by byte correctly
- Convert between string, []byte, and []rune consciously
- Avoid indexing strings when Unicode matters

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Strings are bytes

```
s := "Go👍"
fmt.Println(len(s)) // byte length, not character count
for i, r := range s {
    // r is rune (code point); i is byte index
    fmt.Println(i, string(r))
}
```

A **rune** is an alias for `int32` representing a Unicode code point. UTF-8 encodes code points in 1–4 bytes. Indexing `s[i]` gives a **byte**, not a character.

Conversions

```
b := []byte("hi")
s := string(b)
rs := []rune("naïve")
```

Conversions copy data. Hot paths sometimes stay in `[]byte` to avoid allocations.

Building strings

Prefer `strings.Builder` when concatenating in a loop — repeated `+` allocates often.

Common mistakes (especially from Python instincts)

- Using `len(s)` as character count for Unicode text
- Slicing mid-code-point and creating invalid UTF-8
- Comparing strings with locale rules (Go compares bytes)

Exercises

Work in `exercises/06-strings-runes-bytes/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/06-strings-runes-bytes/` until you have an honest failing attempt.

Self-check

1. Difference between `for i := range s` and `for i, r := range s`?
2. Why is `string` immutable important for concurrency?
3. When use `[]byte` instead of `string`?

Further reading

- https://go.dev/blog/strings
- https://go.dev/blog/normalization